

Prometheus + Grafana



Prometheus is an open-source monitoring and alerting tool designed for reliability and flexibility. Prometheus collects and stores it’s metrics as a time series data i.e. metrics information is stored with timestamp at recorded time.

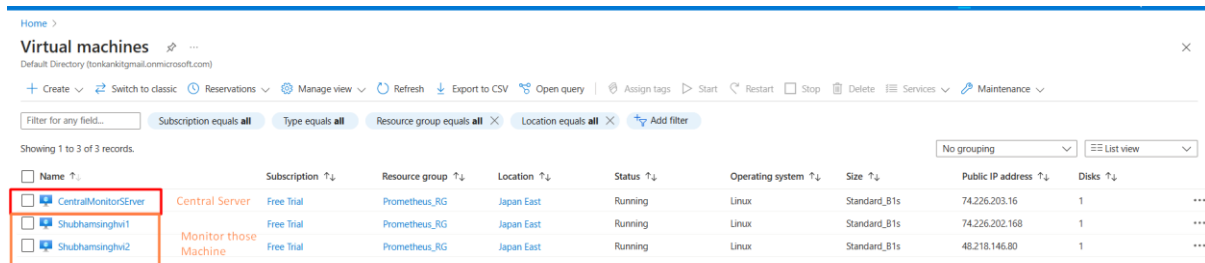
Prometheus is pull Based model means that Prometheus server pulls metrics from target endpoint at specified interval. Prometheus has control over data Scalability , and Transparency Flexible.

Steps of setting up Prometheus from scratch to advance

Download the Prometheus from the official site <https://prometheus.io/download/> for my case I am setting up for Linux Virtual Machine.

2.55.1 / 2024-11-04 Release notes				
File name	OS	Arch	Size	SHA256 Checksum
prometheus-2.55.1.darwin-amd64.tar.gz	darwin	amd64	107.05 MiB	ba915f45b680566646fc824f2b9793d0cd2741c157a5a599be2cb4665b38b498
prometheus-2.55.1.linux-amd64.tar.gz	linux	amd64	106.45 MiB	19700bdd42ec31ee162e4879ebda4cd0ba4432df4daa637141bdbea4b1cd8927
prometheus-2.55.1.windows-amd64.zip	windows	amd64	108.99 MiB	c1a3b8c0c305d7d4d9aaf836425653945a4aad4839bab32114bfe4ad97ff84ba

Through Linux Based System I am monitoring Central Server and two Normal machines using Prometheus



Name	Subscription	Resource group	Location	Status	Operating system	Size	Public IP address	Disks
CentralMonitorServer	Central Server	Free Trial	Prometheus_RG	Japan East	Running	Linux	Standard_B1s	74.226.203.16
Shubhamsinghvi1	Monitor those	Free Trial	Prometheus_RG	Japan East	Running	Linux	Standard_B1s	74.226.202.168
Shubhamsinghvi2	Machine	Free Trial	Prometheus_RG	Japan East	Running	Linux	Standard_B1s	48.218.146.80

Step 1 :

Install Prometheus in system

wget <https://github.com/prometheus/prometheus/releases/download/v2.55.1/prometheus-2.55.1.linux-amd64.tar.gz>

```
shubhamsinghvi@CentralMonitorServer:~$ pwd
/home/shubhamsinghvi
shubhamsinghvi@CentralMonitorServer:~$ wget https://github.com/prometheus/prometheus/releases/download/v2.55.1/prometheus-2.55.1.linux-amd64.tar.gz
--2024-11-12 11:02:01-- https://github.com/prometheus/prometheus/releases/download/v2.55.1/prometheus-2.55.1.linux-amd64.tar.gz
Resolving github.com (github.com)... 20.27.177.113
Connecting to github.com (github.com)|20.27.177.113|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/6838921/3ff37e04-662a-40f4-9da8-514c0aa2fe4c?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20241112%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20241112T110202Z&X-Amz-Expires=300&X-Amz-Signature=d8254dc2817d15719d9d2869ff22a87423863cc1f28de9dcc2da40a218294811&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dprometheus-2.55.1.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream [following]
--2024-11-12 11:02:02-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/6838921/3ff37e04-662a-40f4-9da8-514c0aa2fe4c?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20241112%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20241112T110202Z&X-Amz-Expires=300&X-Amz-Signature=d8254dc2817d15719d9d2869ff22a87423863cc1f28de9dcc2da40a218294811&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dprometheus-2.55.1.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 111626041 (106M) [application/octet-stream]
Saving to: 'prometheus-2.55.1.linux-amd64.tar.gz'

prometheus-2.55.1.linux-amd64.tar.gz  100%[=====] 106.45M  36.5MB/s   in 2.9s

2024-11-12 11:02:05 (36.5 MB/s) - 'prometheus-2.55.1.linux-amd64.tar.gz' saved [111626041/111626041]
```

Step 2 : Unzip downloaded folder and check all files are correct or not.

Tar xvfz <Folder name>

```
shubhamsinghvi@CentralMonitorServer:~$ ls
prometheus-2.55.1.linux-amd64.tar.gz
shubhamsinghvi@CentralMonitorServer:~$ tar xvfz prometheus-2.55.1.linux-amd64.tar.gz
prometheus-2.55.1.linux-amd64/
prometheus-2.55.1.linux-amd64/consoles/
prometheus-2.55.1.linux-amd64/consoles/node-disk.html
prometheus-2.55.1.linux-amd64/consoles/node-overview.html
prometheus-2.55.1.linux-amd64/consoles/prometheus.html
prometheus-2.55.1.linux-amd64/consoles/index.html.example
prometheus-2.55.1.linux-amd64/consoles/node-cpu.html
prometheus-2.55.1.linux-amd64/consoles/prometheus-overview.html
prometheus-2.55.1.linux-amd64/consoles/node.html
prometheus-2.55.1.linux-amd64/promtool
prometheus-2.55.1.linux-amd64/LICENSE
prometheus-2.55.1.linux-amd64/prometheus
prometheus-2.55.1.linux-amd64/prometheus.yml
prometheus-2.55.1.linux-amd64/console_libraries/
prometheus-2.55.1.linux-amd64/console_libraries/menu.lib
prometheus-2.55.1.linux-amd64/console_libraries/prom.lib
prometheus-2.55.1.linux-amd64/NOTICE
shubhamsinghvi@CentralMonitorServer:~$ ls
prometheus-2.55.1.linux-amd64  prometheus-2.55.1.linux-amd64.tar.gz
shubhamsinghvi@CentralMonitorServer:~$ cd prometheus-2.55.1.linux-amd64/
shubhamsinghvi@CentralMonitorServer:~/prometheus-2.55.1.linux-amd64$ ls
LICENSE NOTICE console_libraries consoles prometheus prometheus.yml promtool
shubhamsinghvi@CentralMonitorServer:~/prometheus-2.55.1.linux-amd64$
```

Prometheus is a CLI tool which helps to get the below information

```
-bash: ./prometheus: No such file or directory
shubhamsinghvi@CentralMonitorSErver:~/prometheus-2.55.1.linux-amd64$ ./prometheus --help
usage: prometheus [-flags]

The Prometheus monitoring server

Flags:
  -h, --[no-]help                Show context-sensitive help (also try --help-long and --help-man).
  --[no-]version                 Show application version.
  --config.file="prometheus.yml" Prometheus configuration file path
  --web.listen-address=0.0.0.0:9090 ... Address to listen on for UI, API, and telemetry. Can be repeated.
  --auto-gomemlimit.ratio=0.9 ... The ratio of reserved GOMEMLIMIT memory to the detected maximum container or system memory
  --web.config.file=""           [EXPERIMENTAL] Path to configuration file that can enable TLS or authentication.
  --web.read-timeout=5m         Maximum duration before timing out read of the request, and closing idle connections.
  --web.max-connections=512     Maximum number of simultaneous connections across all listeners.
  --web.external-url=<URL>      The URL under which Prometheus is externally reachable (for example, if Prometheus is served via a reverse proxy). Used for
                                generating relative and absolute links back to Prometheus itself. If the URL has a path portion, it will be used to prefix
                                all HTTP endpoints served by Prometheus. If omitted, relevant URL components will be derived automatically.
  --web.route-prefix=<path>     Prefix for the internal routes of web endpoints. Defaults to path of --web.external-url.
  --web.user-assets=<path>      Path to static asset directory, available at /user.
  --[no-]web.enable-lifecycle   Enable shutdown and reload via HTTP request.
  --[no-]web.enable-admin-api   Enable API endpoints for admin control actions.
  --[no-]web.enable-remote-write-receiver Enable API endpoint accepting remote write requests.
  --web.remote-write-receiver.accepted-protobuf-messages=prometheus.WriteRequest... List of the remote write protobuf messages to accept when receiving the remote writes. Supported values:
                                prometheus.WriteRequest, io.prometheus.write.v2.Request
  --web.console.templates="consoles" Path to the console template directory, available at /consoles.
  --web.console.libraries="console_libraries" Path to the console library directory.
  --web.page-title="Prometheus Time Series Collection and Processing Server" Document title of Prometheus instance.
  --web.cors.origin="*"          Regex for CORS origin. It is fully anchored. Example: 'https://(domain1|domain2)\.com'
  --storage.tsdb.path="data/"
```

We can configure Prometheus using YAML File. In this file we have 3 major blocks of configuration with which we can setup Prometheus monitoring tool.

```
shubhamsinghvi@CentralMonitorSErver:~/prometheus-2.55.1.linux-amd64$ cat prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]
shubhamsinghvi@CentralMonitorSErver:~/prometheus-2.55.1.linux-amd64$ |
```

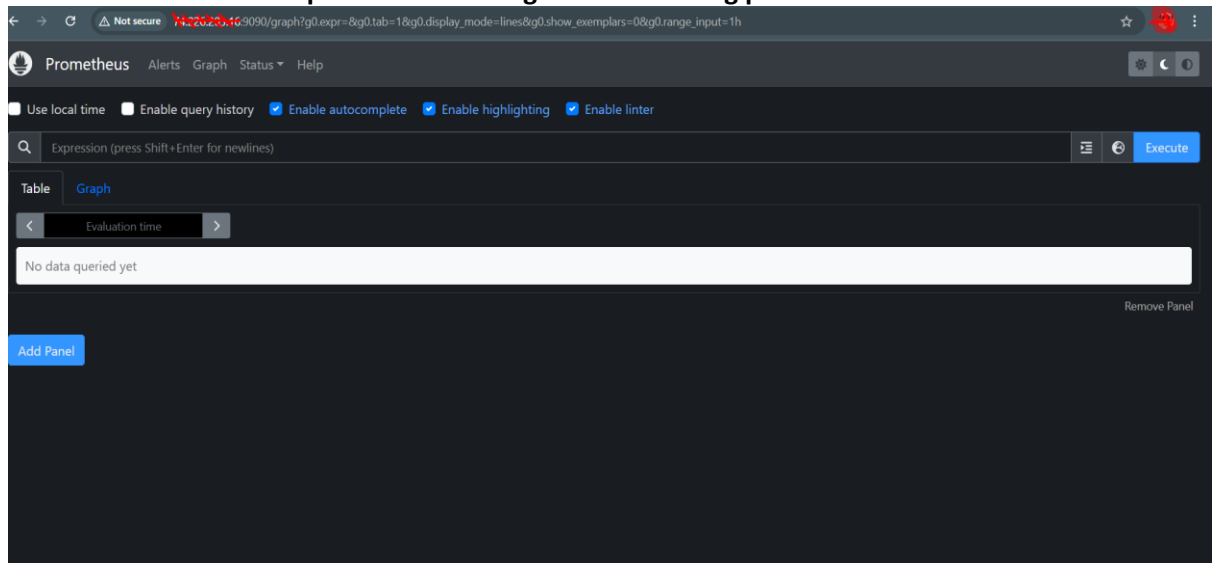
Now we start with our newly created configuration using this command

`./prometheus --config.file=prometheus.yml`

Prometheus should start up. You should also be able to browse to a status page about itself at <http://localhost:9090> in my case I am using my Public IP.

Note : It's not good practice to work as Public IP but for understanding purpose we are doing Public IP.

Now Our Prometheus server is up and for accessing we are enabling port rules for Virtual Machine



Now checking Central server graph and Logs



At that point we monitor central server and other Machines link with Central server
For that let's begin again and monitor other virtual machines.

Monitoring other targets:

https://github.com/prometheus/node_exporter

<https://prometheus.io/docs/guides/node-exporter/>

Download node exporter

https://prometheus.io/download/#node_exporter

As we downloaded Prometheus already , now we will setup node exporter for Other targets

```
shubhamsinghvi@shubhamsinghvi1:~$ pwd
/home/shubhamsinghvi
shubhamsinghvi@shubhamsinghvi1:~$ wget https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
--2024-11-12 17:39:54-- https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
Resolving github.com (github.com)... 20.27.177.115
Connecting to github.com (github.com)|20.27.177.115|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/9524057/a7e04f41-5543-40e2-9060-26fefe32bb4b?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20241112%2Fus-east-1%2F%3Faws4_request&X-Amz-Date=20241112T173954Z&X-Amz-Expires=300&X-Amz-Signature=485dfcd7db9eb1fe36c84a0935370ce5b46841f2e3eb30e90516c5c1cb7e7a&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dnode_exporter-1.8.2.linux-amd64.tar.gz&response-content-type=application%2Foctet-stream [following]
--2024-11-12 17:39:54-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/9524057/a7e04f41-5543-40e2-9060-26fefe32bb4b?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20241112%2Fus-east-1%2F%3Faws4_request&X-Amz-Date=20241112T173954Z&X-Amz-Expires=300&X-Amz-Signature=485dfcd7db9eb1fe36c84a0935370ce5b46841f2e3eb30e90516c5c1cb7e7a&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dnode_exporter-1.8.2.linux-amd64.tar.gz
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.110.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 10676343 (10M) [application/octet-stream]
Saving to: 'node_exporter-1.8.2.linux-amd64.tar.gz'

node_exporter-1.8.2.linux-amd 100%[=====] 10.18M 50.9MB/s in 0.2s

2024-11-12 17:39:55 (50.9 MB/s) - 'node_exporter-1.8.2.linux-amd64.tar.gz' saved [10676343/10676343]
```

Now unzip and check the node exported folder.

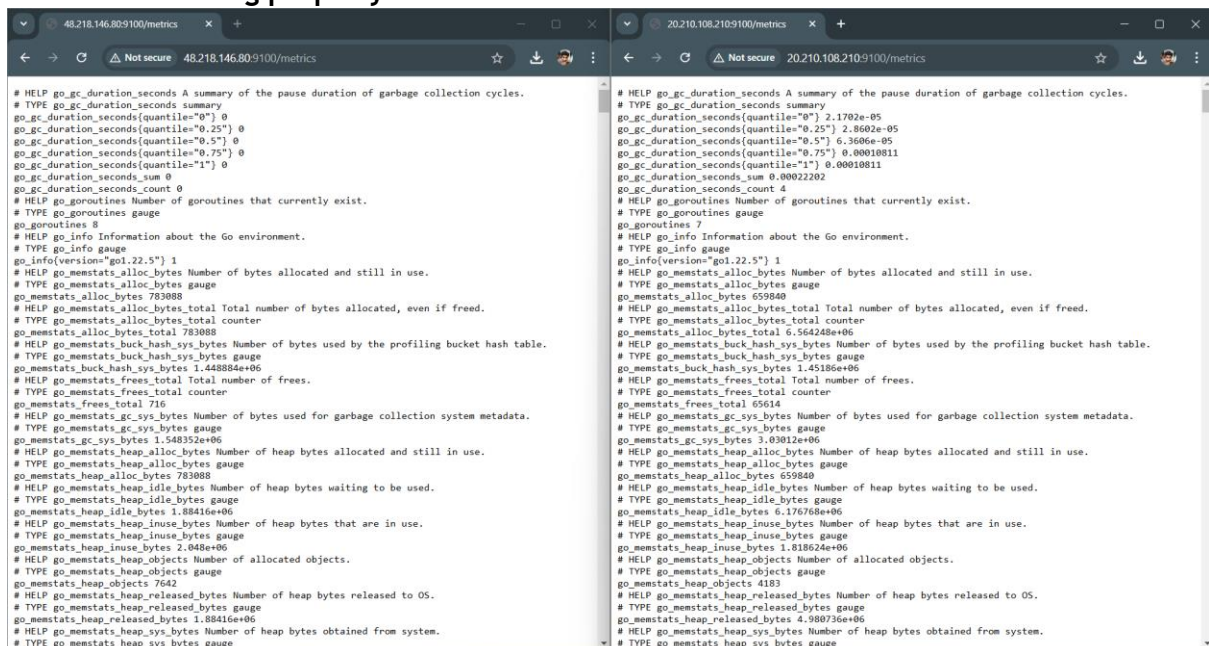
```
shubhamsinghvi@shubhamsinghvi1:~$ ls
node_exporter-1.8.2.linux-amd64.tar.gz
shubhamsinghvi@shubhamsinghvi1:~$ tar xvfz node_exporter-1.8.2.linux-amd64.tar.gz
node_exporter-1.8.2.linux-amd64/
node_exporter-1.8.2.linux-amd64/NOTICE
node_exporter-1.8.2.linux-amd64/node_exporter
node_exporter-1.8.2.linux-amd64/LICENSE
shubhamsinghvi@shubhamsinghvi1:~$ ls
node_exporter-1.8.2.linux-amd64 node_exporter-1.8.2.linux-amd64.tar.gz
shubhamsinghvi@shubhamsinghvi1:~$
```

Once we setup node exported in other Machine then we can enable port 9100 for accessing in node exporter.

```
shubhamsinghvi@shubhamsi x + v x
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=pressure
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=rapl
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=schedstat
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=sellinux
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=sockstat
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=softnet
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=stat
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=tapestats
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=textfile
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=thermal_zone
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=time
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=timex
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=udp_queues
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=uname
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=vmstat
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=watchdog
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=xfs
ts=2024-11-12T17:59:02.389Z caller=node_exporter.go:118 level=info collector=zfs
ts=2024-11-12T17:59:02.391Z caller=tlscfg.go:313 level=info msg="Listening on" address=[::]:9100
ts=2024-11-12T17:59:02.391Z caller=tlscfg.go:316 level=info msg="TLS is disabled." http2=false address=[::]:9100
*[[A[[A

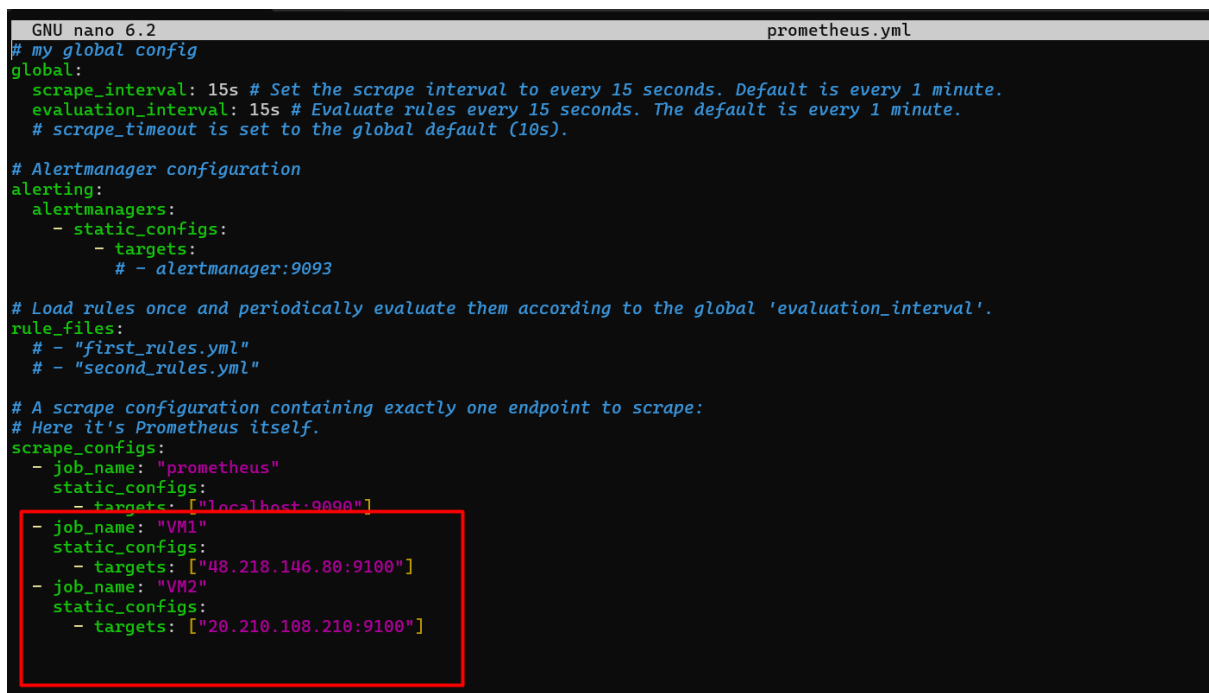
shubhamsinghvi@shubhamsi x + v x
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=pressure
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=rapl
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=schedstat
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=sellinux
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=sockstat
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=softnet
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=stat
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=tapestats
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=textfile
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=thermal_zone
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=time
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=timex
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=udp_queues
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=uname
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=vmstat
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=watchdog
ts=2024-11-12T18:08:12.649Z caller=node_exporter.go:118 level=info collector=xfs
ts=2024-11-12T18:08:12.650Z caller=tlscfg.go:313 level=info msg="Listening on" address=[::]:9100
ts=2024-11-12T18:08:12.651Z caller=tlscfg.go:316 level=info msg="TLS is disabled." http2=false address=[::]:9100
```

once we setup node exporter properly in all virtual machines you can also check all metrics is showing properly



```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 9
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.22.5"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 783088
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 783088
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.448884e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 716
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 1.548352e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 783088
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 1.88416e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 2.048e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 7642
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 1.88416e+06
# HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system.
# TYPE go_memstats_heap_sys_bytes gauge
```

Now our main task is to configure all machine in prometheus server for that we need to work on prometheus.yml file. Let's begin



```
GNU nano 6.2 prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
  - job_name: "VM1"
    static_configs:
      - targets: ["48.218.146.80:9100"]
  - job_name: "VM2"
    static_configs:
      - targets: ["20.210.108.210:9100"]
```

scrape_configs:

- job_name: "prometheus"

static_configs:

- targets: ["localhost:9090"]

- job_name: "VM1"

static_configs:

- targets: ["48.218.146.80:9100"] ## Change your machine IP According

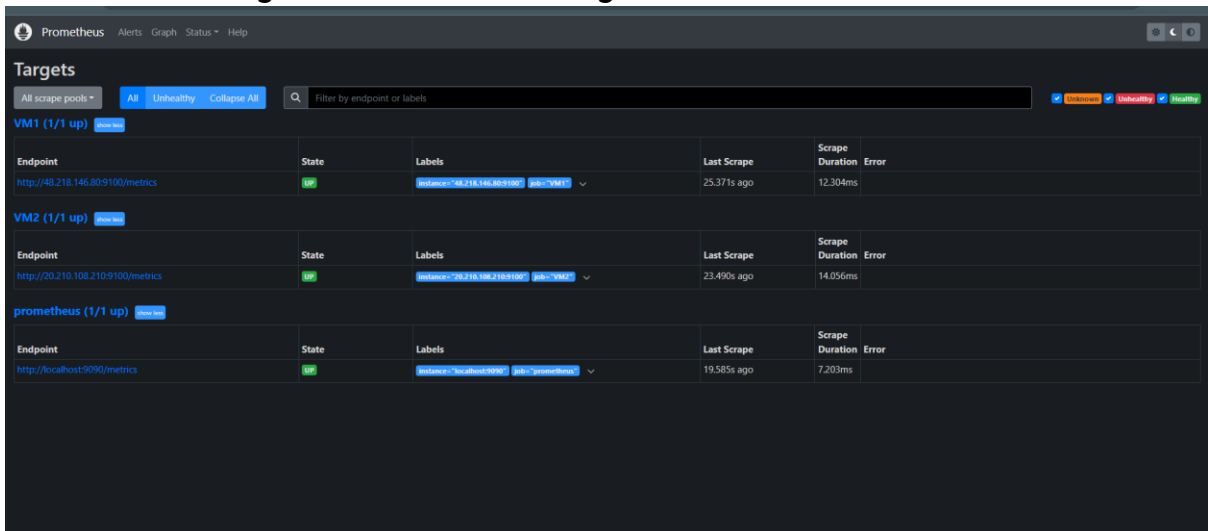
- job_name: "VM2"

static_configs:

- targets: ["20.210.108.210:9100"]] ## Change your machine IP According

Now All working machine is showing with Prometheus server

And all the working machines are in running status



The screenshot shows the Prometheus web interface with the 'Targets' tab selected. It displays three scrape targets, all of which are 'up' and have no errors. The targets are VM1, VM2, and the Prometheus server itself.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
VM1 (1/1 up)					
http://48.218.146.80:9100/metrics	up	instance="48.218.146.80:9100" job="VM1"	25.371s ago	12.304ms	
VM2 (1/1 up)					
http://20.210.108.210:9100/metrics	up	instance="20.210.108.210:9100" job="VM2"	23.490s ago	14.056ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	up	instance="localhost:9090" job="prometheus"	19.585s ago	7.203ms	

Grafana

To install and setup Grafana we need to follow certain rules and instructions

1. Download Prometheus and node_exporter
2. Install Prometheus node_exporter
3. Install and configure Prometheus

In this case we are working with Ubuntu Machine so for Grafana we also setup Linux based system.

<https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/>

- 1- Install the prerequisite packages:

bash

Copy

```
sudo apt-get install -y apt-transport-https software-properties-common wget
```

sudo apt-get install -y apt-transport-https software-properties-common wget

```
shubhamsinghvi@CentralMonitorServer:~$ sudo apt-get install -y apt-transport-https software-properties-common wget
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
software-properties-common is already the newest version (0.99.22.9).
software-properties-common set to manually installed.
wget is already the newest version (1.21.2-2ubuntu1.1).
wget set to manually installed.
The following NEW packages will be installed:
  apt-transport-https
0 upgraded, 1 newly installed, 0 to remove and 3 not upgraded.
Need to get 1510 B of archives.
After this operation, 170 kB of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 apt-transport-https all 2.4.13 [1510 B]
Fetched 1510 B in 0s (65.3 kB/s)
Selecting previously unselected package apt-transport-https.
(Reading database ... 89256 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_2.4.13_all.deb ...
Unpacking apt-transport-https (2.4.13) ...
Setting up apt-transport-https (2.4.13) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

2- Import the GPG key

bash

Copy

```
sudo mkdir -p /etc/apt/keyrings/  
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
```

```
sudo mkdir -p /etc/apt/keyrings/
```

```
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee  
/etc/apt/keyrings/grafana.gpg > /dev/null
```

3- To add a repository for stable releases, run the following command:

bash

Copy

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable  
main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```

4- To add a repository for beta releases, run the following command:

bash

Copy

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com beta main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com beta  
main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
```

5- Run the following command to update the list of available packages:

bash

Copy

```
# Updates the list of available packages  
sudo apt-get update
```

```
sudo apt-get update
```

6- To install Grafana OSS, run the following command:

```
bash
```

[Copy](#)

```
# Installs the latest OSS release:  
sudo apt-get install grafana
```

sudo apt-get install Grafana

7- To install Grafana Enterprise, run the following command:

```
bash
```

[Copy](#)

```
# Installs the latest Enterprise release:  
sudo apt-get install grafana-enterprise
```

sudo apt-get install grafana-enterprise

Once it's setup we need to start Grafana Server for this we need to follow this URL

<https://grafana.com/docs/grafana/latest/setup-grafana/start-restart-grafana/>

Start the Grafana server with systemd

Complete the following steps to start the Grafana server using systemd and verify that it is running.

1 To start the service, run the following commands:

```
bash
```

[Copy](#)

```
sudo systemctl daemon-reload  
sudo systemctl start grafana-server
```

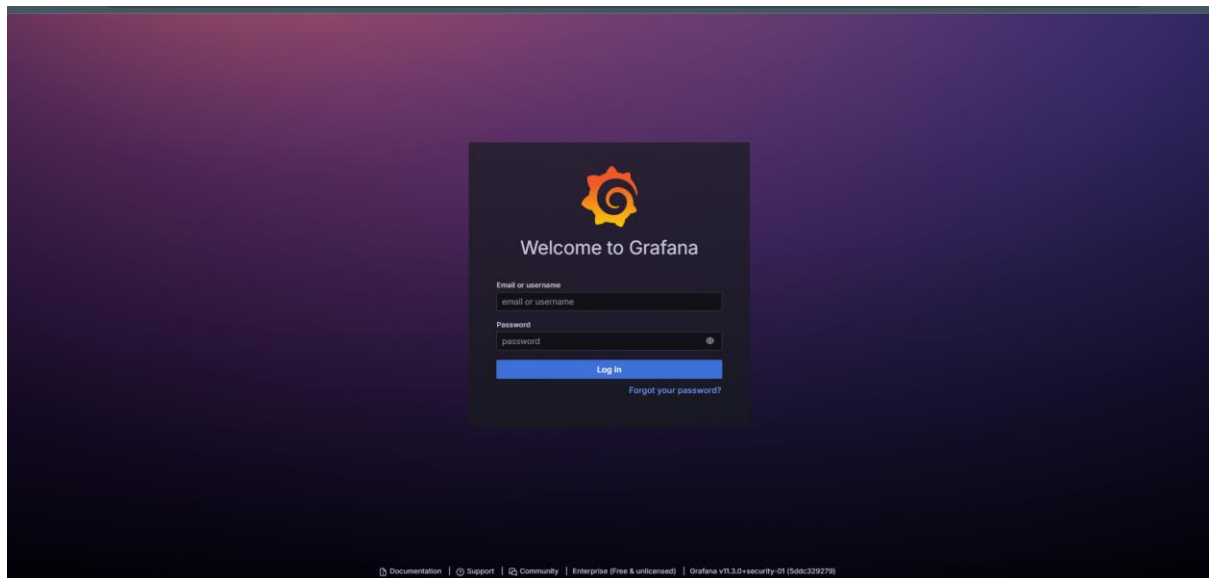
2 To verify that the service is running, run the following command:

```
bash
```

[Copy](#)

```
sudo systemctl status grafana-server
```

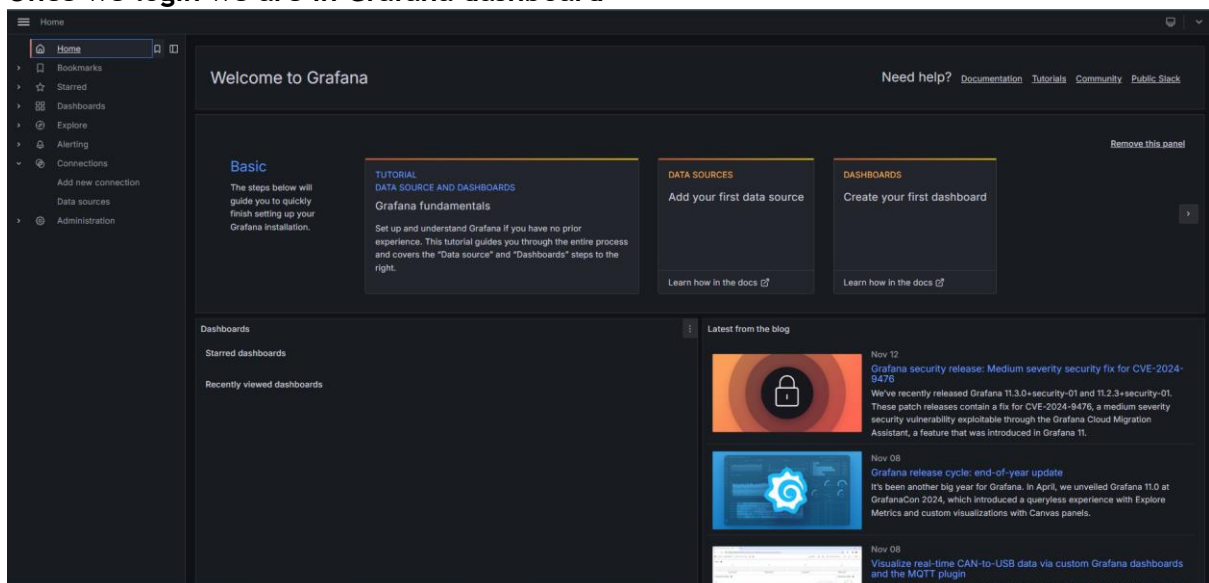
Now our Grafana is setup as you can see in this step .



Now need to login with this portal need to follow that url

<https://grafana.com/docs/grafana/latest/setup-grafana/sign-in-to-grafana/>

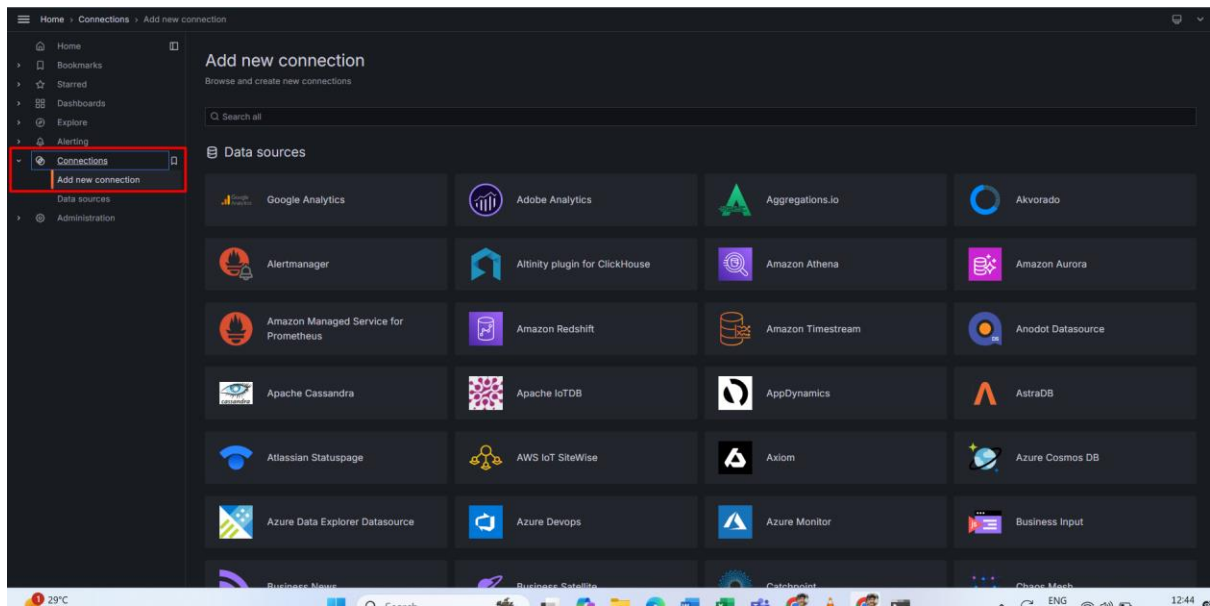
Once we login we are in Grafana dashboard



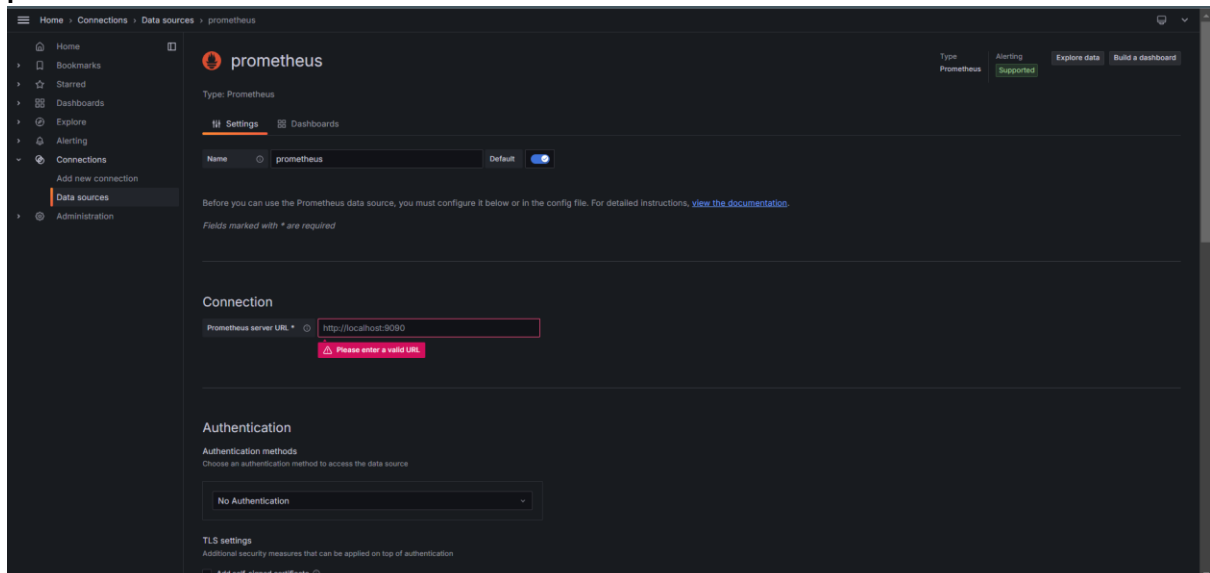
Now we need to connect our Prometheus server to Grafana Dashboard

Add connection to Prometheus server to Grafana follow this path

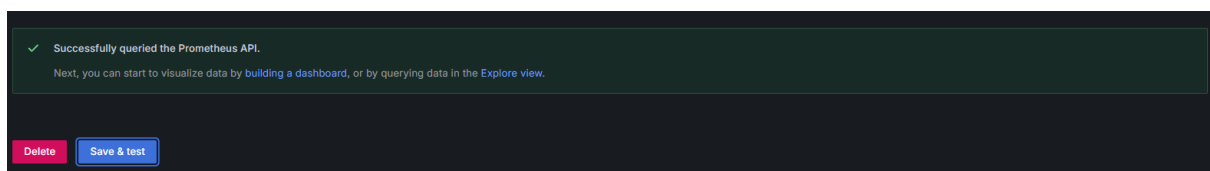
Home > Connection > Add new connection



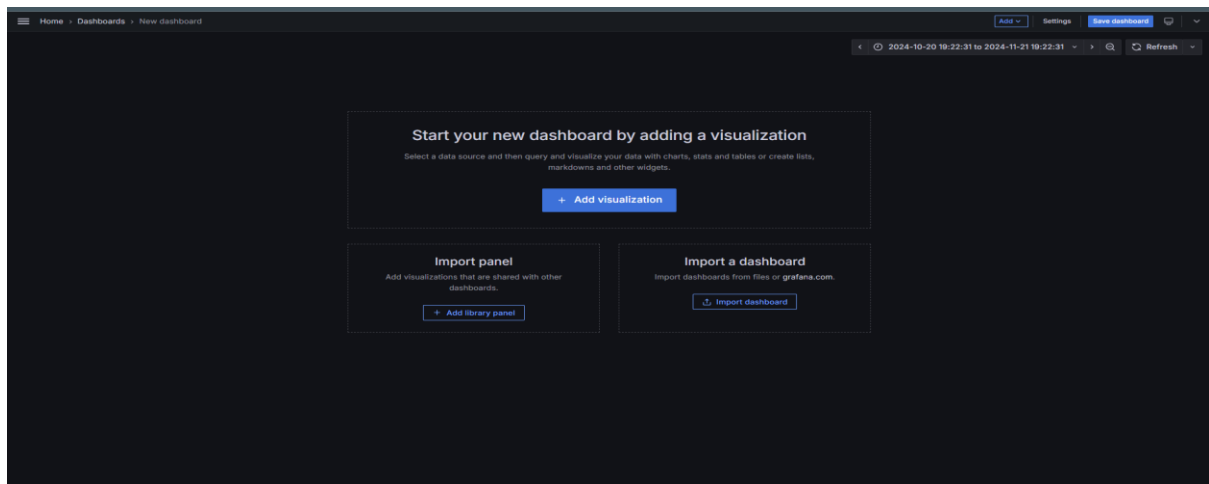
As we know Prometheus is working 9090 port so we can add connection with ip and port



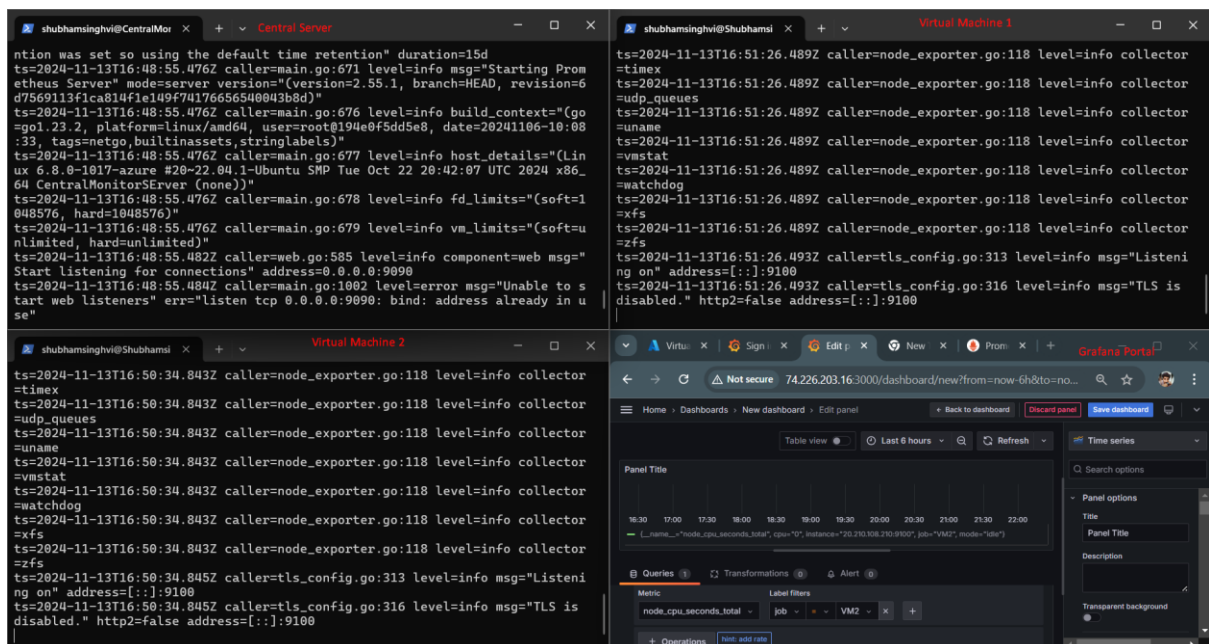
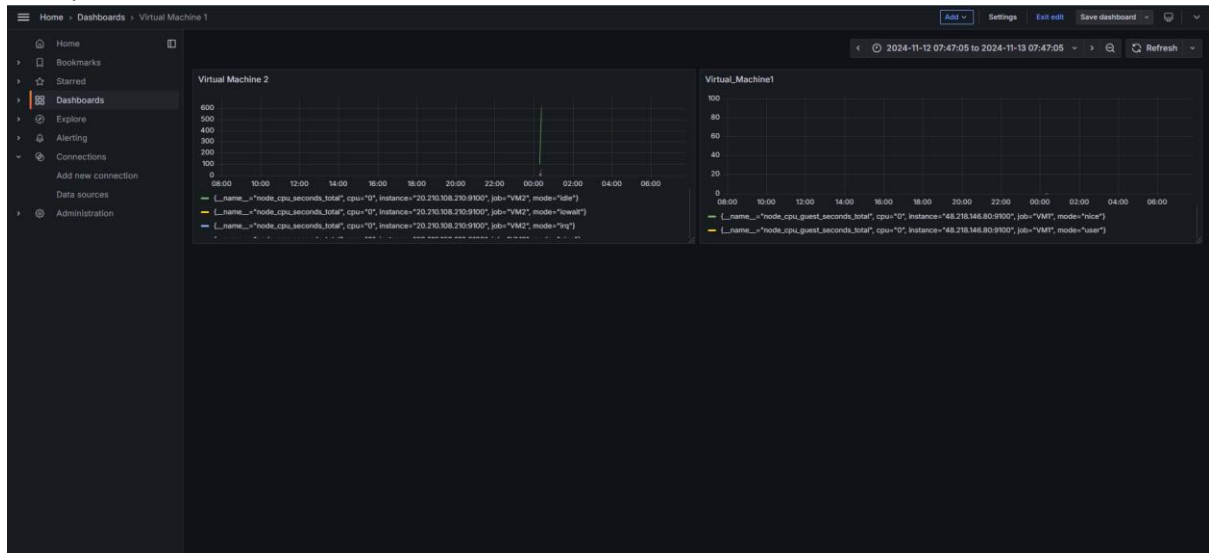
Now we Prometheus successfully connect with Grafana server.



Once we setup everything we need to set Dashboard for our Prometheus server to connect Grafana.



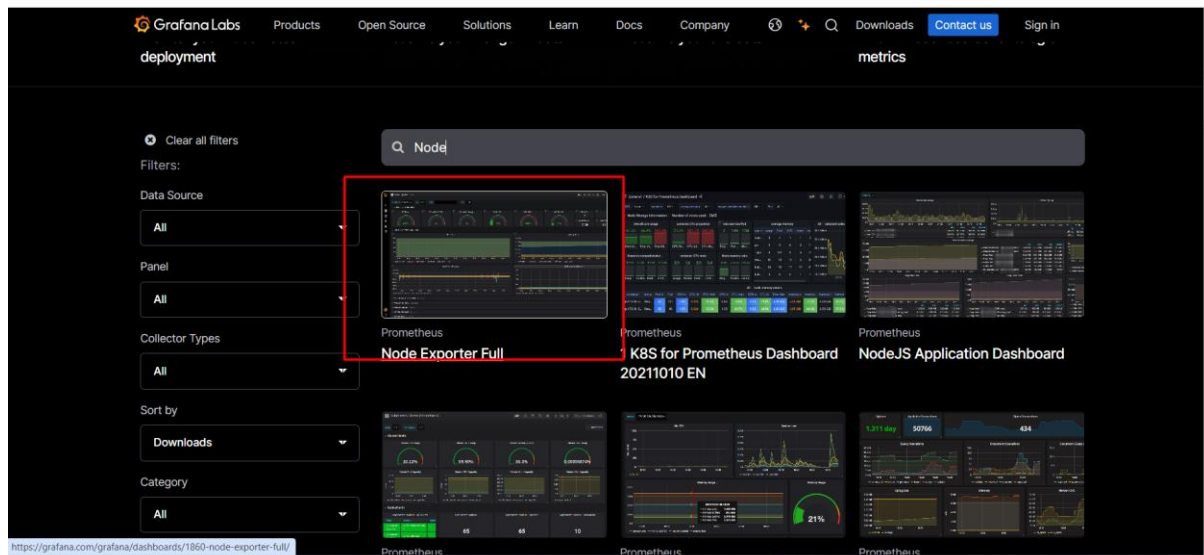
Multiple virtual machine view at same screen



Need to Enhance our dashboard Grafana provide us huge number of dashboard choice for this we need to follow this URL

<https://grafana.com/grafana/dashboards/>

Now currently focusing on Prometheus and node exporter we are searching dashboard according. Import that selected Dashboard in Grafana and check the graph according.



Finally we get the full Dashboard for all connected VM.

